Agentic QoE Optimization for Caching-Aware Cooperative Clustering in Cell-Free Railway Systems

Yu Zhang[1,2], Ziling Xu[3], Shuaifei Chen[4,5*], and Zhenqiao Cheng[6]
[1]State Key Laboratory of Remote Sensing and Digital Earth, Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing 100101, China.
[2]University of Chinese Academy of Sciences, Beijing 100094, China.
[3]School of Electronic and Information Engineering, Beijing Jiaotong University, Beijing 100044, China.
[4]Purple Mountain Laboratories, Nanjing 211111, China.
[5]National Mobile Communications Research Laboratory, School of Information Science and Engineering Southeast University, Nanjing 211189, China.
[6]China Telecom Beijing Research Institute, 6G Research Centre, Beijing 102209, China.
*Corresponding Author: Shuaifei Chen
Emails: zhangyu2311@mails.ucas.ac.cn, xvziling@bjtu.edu.cn, shuaifeichen@seu.edu.cn, chengzq@chinatelecom.cn

2025 IEEE Globecom Workshops (GC Wkshps) | 979-8-3315-6741-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/GCWkshps68340.2025.11590873

***Abstract*—In high-speed railway communications, quality-of-experience (QoE) is challenged by rapidly changing propagation environments and increasing content demands. To this end, we propose an agentic QoE optimization scheme for caching-aware cooperative clustering in cell-free (CF) railway systems. By formulating the problem as a Markov decision process (MDP), a reinforcement learning (RL) agent autonomously perceives user preferences, channel dynamics, and cache states to jointly optimize access point (AP) clustering and content caching. Specifically, the agent employs a soft actor-critic (SAC) architecture to first pre-train an AP clustering policy under varying channel conditions. The resulting clustering behavior is then incorporated into the observation space of a second SAC-based caching policy, enabling the agent to make adaptive content placement decisions based on both real-time user demand profiles and cooperative AP-user associations. Additionally, an alternative demand-aware greedy clustering policy is introduced to better align clustering decisions with personalized content preferences. Extensive simulations demonstrate the scheme's effectiveness, yielding notable QoE gains over state-of-the-art baselines.**

***Index Terms*—Cell-free massive MIMO, cooperative clustering, caching-aware systems, deep reinforcement learning.**

I. Introduction

In high-speed railway systems, ensuring reliable communications is challenging due to the high speeds of the trains and the resulting fluctuating propagation environments [1], which directly impact the quality-of-experience (QoE) of passengers. Traditional cellular systems struggle to maintain continuous service when a user equipment (UE) moves rapidly across coverage areas, leading to frequent handovers and potential service interruptions. Cell-free massive multiple-input-multiple-output (CF mMIMO) [2] offers a promising solution to these challenges. By exploiting access point (AP) cooperative clustering architecture and eliminating traditional handovers, CF mMIMO enhances communication reliability and minimizes delays [3]. Within this framework, APs collaboratively serve UEs simultaneously, improving system performance through macro-diversity and ensuring a more stable connection in a dynamic environment.

However, two significant challenges exist, including the difficulty in effectively meeting the increasing request for content delivery and the absence of efficient policies to adjust AP clustering in CF mMIMO railway networks [4]. Implementing a distributed caching policy to store frequently required files at trackside APs is a promising solution but the dynamic coordination of caching and AP clustering in CF mMIMO railway systems remains inadequately addressed in current research [5]. This coordination challenge constitutes a non-deterministic polynomial-time hard (NP-hard) cooperative resource allocation problem [6], for which deep reinforcement learning (DRL)-based agentic optimization has shown potential [7], e.g., the Deep Deterministic Policy Gradient (DDPG) algorithm [8] has been shown to outperform heuristic approaches such as η-HC [9]. The soft actor-critic (SAC) algorithm [10] has demonstrated remarkable performance in dynamic communication scenarios by providing stable learning in environments with high-dimensional action spaces [11]. However, existing studies still lack realistic agentic environment modeling and effective strategies to handle the excessively large action space problem in such challenges, leading to suboptimal policy performance and slower convergence [12], which motivates this work.

Our major contributions are summarized as follows. We propose an agentic QoE optimization scheme for caching-aware cooperative clustering, along with an individualized Zipf request model tailored for CF mMIMO railway systems, which simulate the intricate processes of communication, storage, and the request mechanism. We formulate a QoE maximization problem and decompose it into clustering and caching optimizations. We propose two approaches for the clustering policy, i.e., a SAC-based method and a channel-aware greedy algorithm. The resulting clustering behavior is integrated into the observation space of a SAC-based caching policy, enabling the agent to adaptively make content placement decisions. Extensive numerical simulations demonstrate that our proposed

This work was supported by the National Natural Science Foundation of China (NSFC) under Grant 62401643 and the China Postdoctoral Science Foundation under Grant 2024M752439.

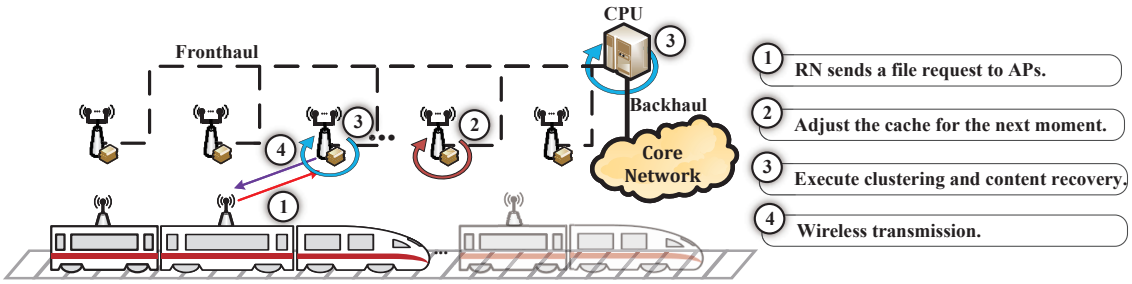

Fig. 1. A caching-aware cooperative clustering framework for CF mMIMO railway systems.

approaches achieve significant improvements over the benchmarks in key performance metrics, including hit probability (HP), spectral efficiency (SE), and QoE.

The rest of this paper is organized as follows. Section II introduces the considered system model. Section III details the problem formulation and solutions exploiting the proposed SAC policies and the greedy policy. Simulation results are provided in Section IV. Finally, Section V draws the conclusions.

II. System Model

We propose a caching-aware cooperative clustering framework for CF mMIMO railway systems, where a train travels through the coverage areas served by a set of track-side multi-antenna APs, each equipped with a local cache of limited capacity c_{ap}. As depicted in Fig. 1, all APs are coordinated by a central processing unit (CPU) via fronthaul connections to facilitate collaborative proactive caching and AP cooperation clustering for joint wireless transmission.

The system contains a train with K carriages that establish communication with the L closest APs. Carriage k, $k \in \{1, \ldots, K\}$, contains U_k users and is equipped with a single-antenna relay node (RN) mounted on the roof. The RN acts as an intermediary, dividing communication into two segments: inboard transmission and outboard transmission. To reasonably simplify the problem, the in-carriage transmission between the RN and the users is assumed to be error-free and delay-free. We only focus on the outboard transmission in this study.

We adopt a standard CF mMIMO wireless communication model [13], treating each carriage as a UE. Exploiting a user-centric framework, at each slot, the selected L APs serve all K UEs in the same time-frequency resource via Time-Division Duplex (TDD) operation, focusing exclusively on the downlink. The coherence block is divided into τ_p channel uses for pilots and τ_d for downlink data, such that $\tau_c = \tau_p + \tau_d$. Since $\tau_p \gg K$, pilot contamination is effectively avoided.

As illustrated in Fig. 1, the entire communication process is divided into the following four steps during each slot within the coherence time:

1) Each UE sends the aggregated requests of all onboard users to the L closest APs through its RN.
2) Upon receiving UE requests, the CPU dynamically updates the caching content of APs for the next slot.
3) The CPU adjusts the AP clustering and each AP cluster reconstructs the requested content from the local cache via fronthaul or from the core network via backhaul.
4) Each AP cluster performs coherent content transmission.

The implementation details of these operations are elaborated in the following sections: the specifics of Step 1) can be found in Section II-A, Steps 2) and 3) are detailed in Section II-B, and Step 4) is covered in Section II-C.

A. Requesting

We propose an individualized Zipf request model to approximate real scenarios. We assume that the statistical distribution of users' file requests, out of a total of F files, follows a Zipf distribution [14], where the probability of requesting a file with frequency ranking f is given by

$$p_f = \frac{(f)^{-\epsilon}}{\sum_{f'=1}^{F} (f')^{-\epsilon}} \tag{1}$$

where p_f is the probability of requesting file f and ϵ is the skewness parameter of Zipf.

More specifically, user u_k in UE k, $u_k \in \{1, \ldots, U_k\}$, independently requests a file from a set of F_{u_k} files, which are a subset of the total F files, with the probability following its individual Zipf distribution. The mapping $f_{u_k} = \gamma_{u_k}(f)$ translates the global frequency ranking f of the total F files into the corresponding frequency ranking f_{u_k} among the specific F_{u_k} files requested by user u_k. The probability of user u_k requesting file f, denoted as $p_{u_k f} = p_{u_k}(f_{u_k})$, is given by

$$p_{u_k}(f_{u_k}) = \begin{cases} \frac{(f_{u_k})^{-\epsilon}}{\sum_{f'_{u_k}=1}^{F_{u_k}} (f'_{u_k})^{-\epsilon}} & \text{, if } f_{u_k} \in \{1, \ldots, F_{u_k}\} \\ 0 & \text{, otherwise} \end{cases} \tag{2}$$

For efficient execution, each file f with size s_f is segmented and provided as ordered chunks for user requests and transmission, while stored as fragments in caching and the core network. Each user requests files according to its individual Zipf distribution, starting from the chunk where the previous request for that file ended last time. If the current chunk is successfully transmitted to the user, the user requests the next chunk in sequence according to the file's chunk order. Once the entire file has been requested, the user starts from the first chunk again if it requests the same file next time. User u_k has a certain level of patience ψ_{u_k}, which decreases by 1 for each slot in which the requested chunk is incomplete. If a chunk fails to be transmitted successfully, the user continues to request that chunk while its patience decreases. If patience reaches zero, the user resets its patience, marks the file's last requested chunk as incomplete, and selects a file according to its individual

Zipf distribution. At slot t, the request of user u_k in UE k is represented by

$$\mathbf{r}_{u_k}^t = [r_{u_k1}, \ldots, r_{u_kF}]^{\mathrm{T}} \in \mathbb{N}^F \tag{3}$$

Each element r_{u_kf} represents the chunk index of file f requested by user u_k, where $r_{u_kf} = 0$ indicates no request. At any slot, only one element in the vector is non-zero, corresponding to the chunk being requested by the user. The total requests in UE k at slot t are denoted by

$$\mathbf{r}_k^t = [r_{k1}, \ldots, r_{kF}]^{\mathrm{T}} \in \mathbb{N}^F \tag{4}$$

Each element r_{kf} represents the total number of requests for the file f in UE k, while n_{kf} denotes the number of distinct chunks requested for the file f in UE k.

B. Caching and Cooperative Clustering

After receiving UE requests, the CPU determines the connection matrix and the caching status of the L serving APs following the established caching and clustering policy elaborated in section III. Each AP has N antennas and serves only one UE at slot t. The connection matrix $\mathbf{D}_{kl}$ is defined as

$$\mathbf{D}_{kl} = \begin{cases} \mathbf{I}_N, & \text{if AP } l \text{ serves UE } k \\ \mathbf{0}_N, & \text{otherwise} \end{cases} \tag{5}$$

indicating whether AP l is serving UE k. The connection matrix $\mathbf{D}_k$ for UE k is then given by

$$\mathbf{D}_k = \mathrm{diag}\left(\mathbf{D}_{k1}, \ldots, \mathbf{D}_{kL}\right) \tag{6}$$

representing the connections of all L APs to UE k.

Prior to the arrival of the train, the serving APs pre-store file fragments according to a pre-determined caching policy, ensuring that the caching is readily available upon request. We denote by $\mathcal{D}_k = \{l : \mathbf{D}_{kl} = \mathbf{I}_N\}$ the set of APs serving UE k. These APs first check whether the required content can be locally reconstructed from their caches. If not, they request additional fragments from the core network to complete the reconstruction. To reduce the complexity of this process, we employ Raptor Codes [15], which allows files to be encoded into fragments and stored across multiple APs. It ensures that, with a sufficient number of fragments, the original file can be reconstructed with a probability of 1, eliminating the need to consider the order of file chunks during caching.

More precisely, let $\boldsymbol{\chi}^t = \{\chi_{lf}^t : l = 1, \ldots, L,\ f = 1, \ldots, F\}$ denote the caching policy at slot t, where $0 \le \chi_{lf}^t \le 1$ indicates the fraction of cache at AP l allocated to the coded fragments of file f, satisfying $\sum_{f=1}^{F} \chi_{lf}^t = 1,\ \forall l, t$. With the help of the Raptor Codes, each AP serving UE k can recover the files once it gathers a sufficient number of fragments and then transmits any required chunk of the file.

The total delay for file reconstruction, associated with acquiring these file chunks, depends on whether the fragments are retrieved from the local cache of the APs or from the core network. The required file f for UE k can be reconstructed using the cached fragments in $\mathcal{D}_k^t$, i.e.,

$$\sum_{l \in \mathcal{D}_k^t} \chi_{lf}^t c_{\mathrm{ap}} \ge s_f^{\mathrm{RC}},\ f = 1, \ldots, F \tag{7}$$

Algorithm 1: Effective Retrieval Delay λ_{lf}^t

Input : $\mathcal{D}_k^t, \boldsymbol{\chi}^t$
Output: λ_{lf}^t

1 Calculate the distance difference between each non-target AP $j \in \mathcal{D}_k^t \setminus \{l\}$ and the target AP l;
2 Allocate the required number of fragments for AP l to recover file f, denoted by $(s_f^{\mathrm{RC}} - \chi_{lf}^t c_{\mathrm{ap}})$, to each AP j based on distance differences, ensuring simultaneous completion under sufficient cache;
3 The available cached fragments decrease as transmission progresses. If an AP j' has insufficient cache, recursively repeat the above steps among the non-target APs in $\mathcal{D}_k^t \setminus \{j'\}$. AP l stops receiving once it has gathered the required fragments, and the time spent constitutes the effective retrieval delay λ_{lf}^t.

where s_f^{RC} denotes the size of file f after Raptor Code encoding. The fronthaul delay is caused by the time it takes to transfer the fragments from the APs within $\mathcal{D}_k^t$. Since the APs perform parallel transmission, the fronthaul delay associated with UE k requesting file f is determined by the AP exhibiting the maximum effective retrieval delay, which is linearly correlated with the product of the required cache fraction and the distance. Specifically, the fronthaul delay is given by

$$\tau_{\mathrm{fh}}^t(k, f) = \delta \cdot \max_{l \in \mathcal{D}_k^t} \left(\lambda_{lf}^t\right) \tag{8}$$

where δ represents the unit transmission delay per fragment per unit distance from the local cache, and the effective retrieval delay λ_{lf}^t, denoting the time required by AP l to gather fragments of file f from other APs in $\mathcal{D}_k^t$, is calculated according to Algorithm 1.

If (7) is not satisfied, the fragments must be fetched from the core network and the backhaul delay is expressed as

$$\tau_{\mathrm{bh}}^t(k, f) = \delta' \left(s_f^{\mathrm{RC}} - \sum_{l \in \mathcal{D}_k^t} \chi_{lf}^t c_{\mathrm{ap}} \right) \tag{9}$$

where δ' represents the unit transmission delay per fragment per unit distance to fetch the remaining uncached fragments from the core network.

The total reconstruction delay $\tau_{\mathrm{re}}^t(k, f)$ is defined as the maximum between the fronthaul delay and the backhaul delay:

$$\tau_{\mathrm{re}}^t(k, f) = \max\left(\tau_{\mathrm{fh}}^t(k, f), \tau_{\mathrm{bh}}^t(k, f)\right) \tag{10}$$

C. Wireless transmission and QoE

In each coherence block, the channels are assumed to be subject to an independent correlated Rayleigh fading. The channel vector between UE k and AP l, denoted as $\mathbf{h}_{kl}$, is modeled as a complex Gaussian random variable as follows,

$$\mathbf{h}_{kl} \sim \mathcal{N}_{\mathbb{C}}\left(\mathbf{0}_N, \mathbf{R}_{kl}\right) \tag{11}$$

where $\mathbf{R}_{kl} \in \mathbb{C}^{N \times N}$ represents the spatial correlation matrix.

During the channel estimation phase, the UE's transmit power ρ_0 remains constant. The received pilot signal at AP

l from UE k is given by

$$\mathbf{y}_{kl}^{\text{pilot}} = \sqrt{\tau_p \rho_0}\mathbf{h}_{kl} + \mathbf{n}_{kl} \tag{12}$$

where $\mathbf{n}_{kl} \sim \mathcal{N}_{\mathbb{C}}\left(\mathbf{0}_N, \sigma^2 \mathbf{I}_N\right)$ represents the additive noise with variance σ^2.

Subsequently, channel estimation is performed using the minimum mean-squared-error (MMSE) method as

$$\widehat{\mathbf{h}}_{kl} = \sqrt{\tau_p \rho_0}\, \mathbf{R}_{kl}\, \mathbf{\Psi}_{kl}^{-1}\, \mathbf{y}_{kl}^{\text{pilot}} \tag{13}$$

where the matrix $\mathbf{\Psi}_{kl}$ is defined as

$$\mathbf{\Psi}_{kl} = \tau_p \rho_0\, \mathbf{R}_{kl} + \sigma^2 \mathbf{I}_N \tag{14}$$

The covariance matrix of the estimated channel is given by

$$\mathbb{E}\left\{\widehat{\mathbf{h}}_{kl}\, \widehat{\mathbf{h}}_{kl}^{\text{H}}\right\} = \mathbf{B}_{kl} = \tau_p \rho_0\, \mathbf{R}_{kl}\, \mathbf{\Psi}_{kl}^{-1}\, \mathbf{R}_{kl} \tag{15}$$

The estimated channel vector $\widehat{\mathbf{h}}_{kl}$ follows a distribution:

$$\widehat{\mathbf{h}}_{kl} \sim \mathcal{N}_{\mathbb{C}}(\mathbf{0}_N, \mathbf{B}_{kl}) \tag{16}$$

During the downlink transmission phase, Maximum Ratio (MR) precoding is applied as follows,

$$\mathbf{w}_{kl} = \sqrt{\rho_{kl}}\bar{\mathbf{w}}_{kl}, \quad \bar{\mathbf{w}}_{kl} = \widehat{\mathbf{h}}_{kl} \tag{17}$$

where the power allocation ρ_{kl} is given by (18) since each AP serves only one UE per coherence block and allocates all power ρ_{t} to the serving UE, i.e.,

$$\rho_{kl} = \begin{cases} \rho_{\text{t}}, & \text{if } \mathbf{D}_{kl} = \mathbf{I}_N \\ 0, & \text{otherwise} \end{cases} \tag{18}$$

Based on the above definitions, the downlink spectral efficiency (SE) for UE k can be expressed in closed form as

$$\text{SE}_k = \frac{\tau_d}{\tau_c} \log_2\left(1 + \text{SINR}_k^{\text{dl}}\right) \tag{19}$$

where $\text{SINR}_k^{\text{dl}}$ is computed as

$$\frac{\sum_{l=1}^{L} \left|\sqrt{\rho_{kl}}\text{tr}\left(\mathbf{D}_{kl}\mathbf{B}_{kl}\right)\right|^2}{\sum_{i=1}^{K}\sum_{l=1}^{L} \rho_{il}\text{tr}\left(\mathbf{D}_{il}\mathbf{B}_{il}\mathbf{D}_{il}\mathbf{R}_{kl}\right) - \sum_{l=1}^{L}\left|\sqrt{\rho_{kl}}\text{tr}\left(\mathbf{D}_{kl}\mathbf{B}_{kl}\right)\right|^2 + \sigma_{\text{dl}}^2} \tag{20}$$

where σ_{dl} denotes the downlink noise. The wireless delay for the transmission of file f to UE k at slot t is then given by

$$\tau_{\text{wl}}^t(k,f) = \frac{n_{kf}}{B \times \text{SE}_k^t} \tag{21}$$

where B denotes the bandwidth. Then, the total delay for transmitting file f to UE k is computed as

$$\tau_{\text{tot}}^t(k,f) = \tau_{\text{wl}}^t(k,f) + \tau_{\text{re}}^t(k,f) \tag{22}$$

To improve the QoE of unserved users, we define $\mathbf{u}_k^t = \{u_{k1}^t, \ldots, u_{kF}^t\}$, where $u_{kf}^t = r_{kf}^t + \omega_{kf}^t \in \mathbb{N}$, and ω_{kf}^t is statistically determined by the weight $\omega_{u_k}^t$ for requests where $r_{u_k f} \neq 0$ in UE k. The weight $\omega_{u_k}^t$ is updated as follows,

$$\omega_{u_k}^t = \begin{cases} \omega_{u_k}^{t-1} + 0.1\Delta\psi_{u_k}^t, & \text{if } u_k \text{ is not served,} \\ 0, & \text{otherwise} \end{cases} \tag{23}$$

where $\Delta\psi_{u_k}^t$ is the change in the patience of user u_k.

During the transmission phase for UE k, the total delay $\tau_{\text{tot}}^t(k,f)$ is calculated for each file f. If the total delay exceeds the remaining coherence time, skip f and continue with the next file in the priority order. Finally, files that can be fully transmitted within the coherence time are defined as $\mathcal{F}_k$. Then, the QoE is calculated as

$$\text{QoE}_k^t = \sum_{f \in \mathcal{F}_k^t}\left(\omega_{kf}^{t-1} + r_{kf}^t\right) \Bigg/ \sum_{f}\left(\omega_{kf}^{t-1} + r_{kf}^t\right) \tag{24}$$

We also calculate the HP as

$$\text{HP}^t = \frac{\sum_{k=1}^{K}\sum_{f=1}^{F} r_{kf}^t \min\left(\frac{c_{\text{ap}}\sum_{l \in \mathcal{D}_k^t}\chi_{lf}^t}{s_f^{\text{RC}}}, 1\right)}{\sum_{k=1}^{K}\sum_{f=1}^{F} r_{kf}^t} \tag{25}$$

III. Problem Formulation and Solution

The optimization problem to maximize the QoE can be formulated as follows,

$$\begin{aligned} \underset{\{\boldsymbol{\chi}^t, \mathbf{D}_k^t\}}{\text{maximize}} \quad & \lim_{T\to\infty}\frac{1}{T}\sum_{t=1}^{T}\sum_{k=1}^{K}\text{QoE}_k^t(\boldsymbol{\chi}^t, \mathbf{D}_k^t) \\ \text{s.t.} \quad & \sum_{f=1}^{F}\chi_{lf}^t \leq 1, \chi_{lf}^t \in [0,1], \\ & \sum_{k=1}^{K}\mathbf{D}_{kl}^t = \mathbf{I}_N, \\ & \mathbf{D}_{kl}^t \in \{\mathbf{I}_N, \mathbf{0}_N\}, \\ & \forall t,\ k = 1, \ldots, K,\ l = 1, \ldots, L,\ f = 1, \ldots, F. \end{aligned} \tag{26}$$

which is non-convex and NP-hard. To solve this problem, we propose three policies: SAC caching, SAC clustering, and greedy clustering. The combination of caching and clustering policies serves as the foundation for constructing the optimal execution strategy in the system.

After perceiving the state information, the agent interacts with the environment through actions, states, and rewards within the framework of a Markov Decision Process (MDP), and employs the policies to maximize the QoE. In our system, the state and action are defined as follows:

- State ($\mathbf{S}_t$): Spatial correlation matrix $\mathbf{R}^t = \{\mathbf{R}_{kl}^t : k = 1, \ldots, K, l = 1, \ldots, l\}$, channel covariance matrix $\mathbf{B}^t = \{\mathbf{B}_{kl}^t : k = 1, \ldots, K, l = 1, \ldots, l\}$, caching matrix $\boldsymbol{\chi}^t$, and weighted user request matrix $\mathbf{U}^t = \{\mathbf{u}_k^t : k = 1, \ldots, K\}$.
- Action ($\mathbf{A}_t$): Caching policy for the next slot $\boldsymbol{\chi}^{t+1}$ and connection identification matrix $\mathbf{D}^t = \{\mathcal{D}_k^t : k = 1, \ldots, K\}$.
- Reward (γ_t): QoE aggregated for all users.

Training the SAC policy within this MDP framework comprehensively accounts for the collaboration between AP caching and channel conditions to meet user requests but leads to challenges in model fitting due to the large action space. Thus, we decompose the problem into two approximate MDPs:

- **Clustering MDP**:
  1) **State ($\mathbf{S}_t$)**: $\mathbf{B}^t$ and $\mathbf{R}^t$.
  2) **Action ($\mathbf{A}_t$)**: $\mathbf{D}^t$.
  3) **Reward (γ_t)**: The mean SE of all users.

Algorithm 2: Greedy Clustering Policy

Input : $\mathbf{B}^t$, $\mathbf{u}^t$
Output: $\mathbf{D}^t$
Initialize $\mathbf{D}^t$ as zeros;
Normalize: $\mathbf{u}^t_{\text{norm}} \leftarrow \mathbf{u}^t / \sum_{k=1}^{K} u^t_k$;
Sort UEs by $\mathbf{u}^t_{\text{norm}}$ in descending order;
foreach *UE k in sorted order* **do**
    Compute the normalized gain vector for UE k: $\mathbf{G}^t_k \leftarrow \{\text{tr}(\mathbf{B}^t_{kl}) : l = 1, \ldots, L\} / \sum_{l=1}^{L} \text{tr}(\mathbf{B}^t_{kl})$;
    Sort APs by $\mathbf{G}^t_k$;
    while $\mathbf{u}^t_{norm}[k] > 0$ *and APs remain* **do**
        Connect the AP j with the highest gain;
        Update $\mathbf{D}^t$ and mark AP j as connected;
        $\mathbf{u}^t_{\text{norm}}[k] \leftarrow \mathbf{u}^t_{\text{norm}}[k] - \mathbf{G}^t_k[j]$;
if *any unconnected APs remain* **then**
    Randomly assign remaining APs to UEs with unmet requests;

- **Caching MDP**:
    1) **State ($\mathbf{S}_t$)**: $\mathbf{U}^t$.
    2) **Action ($\mathbf{A}_t$)**: $\boldsymbol{\chi}^{t+1}$.
    3) **Reward** (γ_t): Treat the clustering of APs as part of the environment to optimize caching, such that the reward is the mean QoE of all users.

Due to the predictability of the train's trajectory, we first train a SAC network within the clustering MDP framework, obtaining the optimal clustering policy based on channel conditions for each position. Subsequently, we treat the clustering of APs as part of the environment and train another SAC network within the caching MDP framework, adapting the SAC caching policy to the clustering policy at each position. However, the SAC clustering policy does not perfectly adapt to the dynamic changes in UE requests, resulting in persistent fluctuations in performance. To achieve better alignment with user requests, we propose a greedy policy.

The greedy clustering policy shown in Algorithm 2 selects APs that offer the greatest immediate boost in signal strength. This approach allocates AP connections to enhance signal gain while ensuring improved alignment with aggregated user requests, denoted by $\mathbf{u}^t = \{u^t_k = \sum_{f=1}^{F} u^t_{kf} : k = 1, \ldots, K\}$.

IV. Numerical Results

In this section, we evaluate the performance of the proposed policies against the benchmark η-HC policy and DDPG policy.

A. Setups

We consider a scenario with 200 users distributed across $K = 4$ carriages with a length of 25 meters on a train traveling along a 30000-meter railway at a speed of 100 m/s, served by $L = 15$ APs per slot. The users request services following a Zipf distribution with a skewness parameter of $\epsilon = 1.5$, and each user selects $F_{u_k} = 40$ files from a total pool of $F = 100$ files, with file sizes ranging from 40 to 60 Mbits. There are 300 APs positioned along the railway equipped with $N = 5$ antennas, arranged in a half-wavelength spaced uniform linear array, and each has a local cache capacity of 250 Mbits.

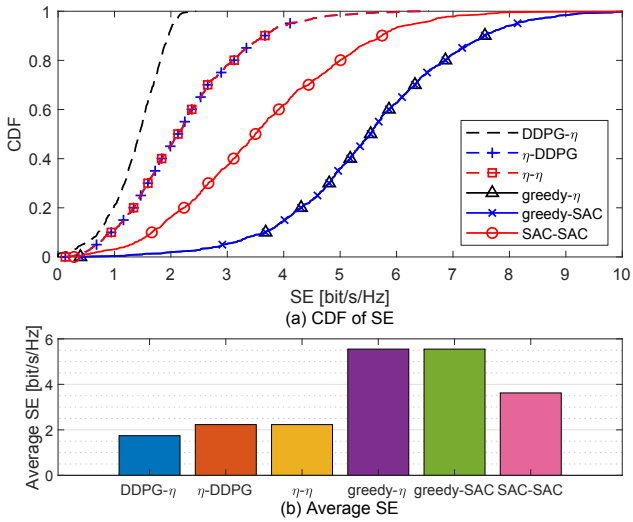

Fig. 2. Comparison of CDF and average values of SE.

For wireless transmissions, we adopt the 3GPP Suburban Microcell model to determine propagation conditions [16]. The downlink power ρ_t is set to 1000 mW, and each UE transmits with 100 mW (ρ_0). The coherence time per slot is 1 ms.

For an episode spanning a 30000-meter railway, we simulate 3×10^5 slots. The clustering network is trained for 1.5×10^7 steps (50 updates per slot). Using either the learned clustering policy or the greedy policy, the caching network is separately trained for 3×10^5 steps and evaluated with 2000 samples. Both networks adopt a four-layer MLP with 512 units per layer and a learning rate of 10^{-6}, ensuring stable training. The η-HC policy reserves 95% of each AP's cache for the most popular fragments ($\eta = 0.95$).

B. Results and discussions

We present the results for six strategies in the following order: 1) DDPG-η: DDPG clustering and η-HC caching policy; 2) η-DDPG: η-HC clustering and DDPG caching policy; 3) η-η: η-HC clustering and η-HC caching policy; 4) greedy-η: greedy clustering and η-HC caching policy; 5) greedy-SAC: greedy clustering and SAC caching policy; 6) SAC-SAC: SAC clustering and SAC caching policy.

During training, the SAC clustering policy exhibits faster convergence. Within the environment, which simulates the intricate processes of communication, storage, and the request mechanism, DDPG encounters challenges in learning the optimal policy due to the large action space. As shown in Fig. 2, the average SE of strategy 6), achieves a twofold improvement compared to benchmark strategies 2) and 3), and a 2.5-fold improvement over benchmark strategy 1). The greedy clustering policy in strategies 4) and 5) derives the clustering policy directly based on channel gain, resulting in an average SE nearly three times that of benchmark strategies 2) and 3). As shown in Fig. 3, strategies 5) and 6), which employ the SAC caching policy, significantly improve prediction accuracy

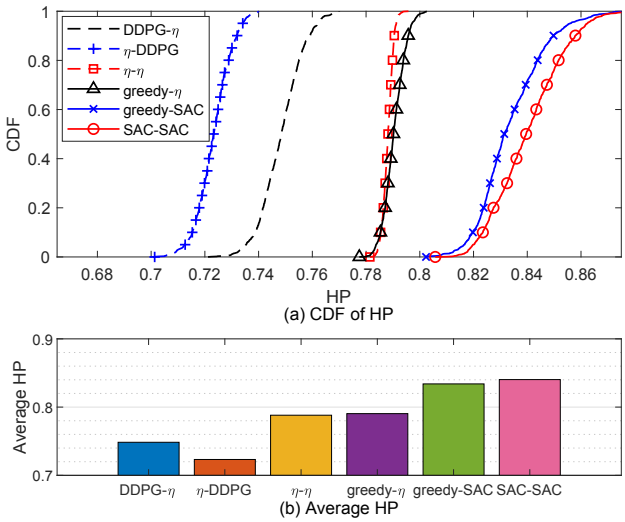

Fig. 3. Comparison of CDF and average values of HP.

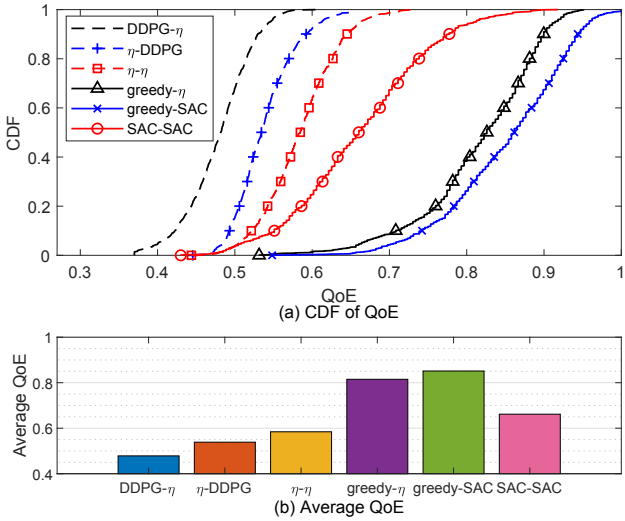

Fig. 4. Comparison of CDF and average values of QoE.

compared to strategies 1), 3), and 4). The poor convergence of DDPG directly leads to reduced prediction accuracy and results in suboptimal clustering strategies that further degrade the prediction performance. Finally, we compare the QoE performance of the six strategies in Fig. 4. As mentioned earlier, strategies 4) and 5), which incorporate the greedy clustering policy, significantly enhance the SE by achieving better alignment with user demands and effectively improve the average QoE compared to the benchmark. Strategy 5) shows that our scheme effectively integrates accurate preference prediction with efficient cache-based wireless transmission, improving QoE.

V. Conclusions

In this paper, we propose an agentic QoE optimization scheme for caching-aware cooperative clustering in CF mMIMO railway systems and simulate the wireless content delivery process via an individualized Zipf request model. The agent autonomously perceives channel dynamics, user preferences, and cache states to jointly optimize clustering and caching decisions. Two clustering policies are introduced, i.e., a SAC-based learning method and a channel-aware greedy algorithm. A SAC-based caching policy is then trained with the clustering behavior embedded into the environment. Numerical results demonstrate that this decomposition approach addresses the challenges posed by extensive action spaces. The SAC caching policy significantly enhances HP, while the SAC clustering policy notably improves SE. The greedy clustering policy achieves better alignment with user demands, further boosting SE and overall QoE. The proposed scheme successfully integrates accurate preference prediction with efficient cache-based wireless transmission, yielding substantial QoE gains.

References

[1] H. Zhang, X. Wang, J. Tan, and J. Wang, "Modem optimization of high-mobility scenarios: A deep-learning-inspired approach," in *ICC Workshops*, June 2024, pp. 1779–1784.

[2] S. Chen, C.-X. Wang, J. Li, C. Huang, H. Chang, Y. Huang, J. Huang, and Y. Chen, "Channel map-based angle domain multiple access for cell-free massive MIMO communications," *IEEE J. Sel. Top. Signal Process.*, vol. 19, no. 2, pp. 366–380, Mar. 2025.

[3] S. Chen, J. Zhang, E. Björnson, J. Zhang, and B. Ai, "Structured massive access for scalable cell-free massive MIMO systems," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 4, pp. 1086–1100, Apr. 2021.

[4] J. Zheng, J. Zhang, H. Du, D. Niyato, B. Ai, M. Debbah, and K. B. Letaief, "Mobile cell-free massive MIMO: Challenges, solutions, and future directions," *IEEE Wirel. Commun.*, vol. 31, no. 3, pp. 140–147, Mar. 2024.

[5] F. Tan, Y. Peng, and Q. Liu, "Cluster content caching: A deep reinforcement learning approach to improve energy efficiency in cell-free massive MIMO networks," *Sensors*, vol. 23, no. 19, p. 8295, Oct. 2023.

[6] C. Zhong, M. C. Gursoy, and S. Velipasalar, "A deep reinforcement learning-based framework for content caching," in *CISS*, Mar. 2018, pp. 1–6.

[7] Y. Wei, F. R. Yu, M. Song, and Z. Han, "Joint optimization of caching, computing, and radio resources for fog-enabled iot using natural actor-critic deep reinforcement learning," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 2061–2073, Oct. 2018.

[8] N. Ghiasi, S. Mashhadi, S. Farahmand, S. M. Razavizadeh, and I. Lee, "Energy efficient AP selection for cell-free massive MIMO systems: Deep reinforcement learning approach," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 1, pp. 29–41, Mar. 2022.

[9] S. Chen, J. Zhang, E. Björnson, S. Wang, C. Xing, and B. Ai, "Wireless caching: Cell-free versus small cells," in *ICC*, June 2021, pp. 1–6.

[10] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *ICML*, July 2018, pp. 1587–1596.

[11] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *ICML*, July 2018, pp. 1861–1870.

[12] G. Zhou, W. Tian, R. Buyya, R. Xue, and L. Song, "Deep reinforcement learning-based methods for resource scheduling in cloud computing: A review and future directions," *Artif. Intell. Rev.*, vol. 57, no. 5, p. 124, May 2024.

[13] E. Björnson and L. Sanguinetti, "Scalable cell-free massive MIMO systems," *IEEE Trans. Commun.*, vol. 68, no. 7, pp. 4247–4261, July 2020.

[14] L. Breslau, P. Cao, L. Fan, G. Phillips, and S. Shenker, "Web caching and zipf-like distributions: Evidence and implications," in *INFOCOM*, Mar. 1999, pp. 126–134.

[15] A. Shokrollahi, "Raptor codes," *IEEE Trans. Inf. Theory*, vol. 52, no. 6, pp. 2551–2567, June 2006.

[16] E. Björnson and L. Sanguinetti, "Making cell-free massive MIMO competitive with MMSE processing and centralized implementation," *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 77–90, Jan. 2019.